Projet M1 Optimisation 2026
Optimisation d'un problème mixQUBO

Abdelmoughit DOUIMIA

Mai 2026

Table des matières

1 Contexte **2**
1.1 Problème QUBO classique 2
1.2 Problème mixQUBO 2
1.3 Interprétation 2

2 Méthodologie **2**

3 Algorithmes implémentés **3**
3.1 Optimisation combinatoire 3
3.1.1 Recherche aléatoire 3
3.1.2 Hill Climbing 3
3.1.3 Recuit simulé 3
3.2 Optimisation continue 3
3.2.1 Recherche aléatoire continue 3
3.2.2 Recherche locale continue 4
3.2.3 Recuit simulé continu 4
3.3 Optimisation mixte 4
3.3.1 Recherche aléatoire mixte 4
3.3.2 Méthode alternée 4
3.3.3 Recuit simulé mixte 4

4 Résultats expérimentaux **4**
4.1 Analyse 5

5 Conclusion **6**
5.1 Perspectives 6

1 Contexte

1.1 Problème QUBO classique

Le problème QUBO (Quadratic Unconstrained Binary Optimization) consiste à minimiser une fonction quadratique définie sur des variables binaires $x \in \{0,1\}^n$. Il s'agit d'un problème combinatoire difficile, car l'espace de recherche augmente exponentiellement avec le nombre de variables binaires.

1.2 Problème mixQUBO

Le problème mixQUBO étend le QUBO classique en introduisant des variables continues en plus des variables binaires.

On considère donc :

— $x \in \{0,1\}^n$: variables binaires

— $z \in [-1,1]^d$: variables continues

La fonction objectif est définie sous la forme suivante :

$$f(x,z) = \sum_k J_k \cdot h(z_u, z_v) \cdot (-1)^{x_i} \cdot (-1)^{x_j} \tag{1}$$

L'objectif du problème est de minimiser cette fonction :

$$\min_{x,z} f(x,z) \tag{2}$$

1.3 Interprétation

Le problème combine deux types d'espaces de recherche :

— un espace combinatoire lié aux variables binaires x

— un espace continu lié aux variables réelles z

Cette interaction rend le problème plus difficile qu'un QUBO classique. En effet, une bonne solution doit à la fois choisir une configuration binaire efficace et ajuster correctement les variables continues.

2 Méthodologie

Les algorithmes ont été testés sur les 16 instances fournies dans le projet, allant de `mxqubo_v0_0.json` à `mxqubo_v0_15.json`.

Le temps d'exécution a été calibré à l'aide du programme `etalon`. Sur notre machine, le budget utilisé est d'environ 31563 ms par exécution, ce qui correspond à environ 15 secondes sur la machine étalon.

Chaque algorithme stochastique a été exécuté 30 fois sur chaque instance afin d'obtenir une évaluation statistiquement plus fiable.

Les indicateurs utilisés pour comparer les algorithmes sont :

— la meilleure valeur finale obtenue, notée `best_f`
— la moyenne des valeurs obtenues
— l'écart-type des valeurs obtenues
— le nombre d'évaluations de la fonction objectif
— le temps d'exécution

Les résultats ont été enregistrés dans un fichier `results.csv`, puis analysés avec Python à l'aide des bibliothèques `pandas` et `matplotlib`.

3 Algorithmes implémentés

3.1 Optimisation combinatoire

Dans cette partie, les variables continues sont fixées à zéro : $z = 0$. L'objectif est donc d'optimiser uniquement les variables binaires x.

3.1.1 Recherche aléatoire

La recherche aléatoire génère des solutions binaires aléatoires et conserve la meilleure solution rencontrée. Cette méthode sert de baseline simple.

3.1.2 Hill Climbing

Le Hill Climbing est une recherche locale qui explore le voisinage d'une solution en inversant un bit. Dans l'implémentation réalisée, le meilleur voisin est choisi lorsque celui-ci améliore la solution courante. Un redémarrage aléatoire est effectué lorsqu'aucune amélioration n'est possible.

3.1.3 Recuit simulé

Le recuit simulé modifie aléatoirement une variable binaire et accepte toujours une amélioration. Il peut aussi accepter une solution moins bonne avec une probabilité dépendant de la température. Cette stratégie permet d'échapper à certains optima locaux.

3.2 Optimisation continue

Dans cette partie, les variables binaires sont fixées à zéro : $x = 0$. L'objectif est donc d'optimiser uniquement les variables continues z.

3.2.1 Recherche aléatoire continue

Cette méthode tire aléatoirement les variables continues dans l'intervalle $[-1, 1]$ et conserve la meilleure solution.

3.2.2 Recherche locale continue

La recherche locale continue modifie légèrement une coordonnée de z à l'aide d'une perturbation gaussienne. La solution est acceptée uniquement si elle améliore la fitness.

3.2.3 Recuit simulé continu

Le recuit simulé continu utilise le même principe que le recuit simulé combinatoire, mais la perturbation est appliquée aux variables continues.

3.3 Optimisation mixte

Dans cette partie, les variables binaires et continues sont optimisées simultanément.

3.3.1 Recherche aléatoire mixte

La recherche aléatoire mixte génère à la fois une solution binaire aléatoire et un vecteur continu aléatoire. Elle sert de baseline pour le problème complet.

3.3.2 Méthode alternée

La méthode **mixed_alternating** alterne deux phases :
— une phase d'optimisation binaire par Hill Climbing
— une phase d'optimisation continue par perturbations locales
Cette méthode exploite séparément les deux structures du problème tout en les combinant dans une même solution.

3.3.3 Recuit simulé mixte

La méthode **mixed_simulated_annealing** modifie aléatoirement :
— une variable binaire
— une variable continue
— ou les deux simultanément
L'acceptation des solutions suit le principe du recuit simulé. Cette méthode permet une exploration plus globale de l'espace de recherche mixte.

4 Résultats expérimentaux

Les expérimentations finales ont été réalisées avec :
— 9 algorithmes
— 16 instances
— 30 exécutions par instance

Le fichier final `results.csv` contient donc 4321 lignes, correspondant à :

$$1 + 9 \times 16 \times 30 = 4321 \tag{3}$$

La figure suivante présente la moyenne de `best_f` pour chaque algorithme. Les barres d'erreur correspondent à l'écart-type calculé sur les exécutions.

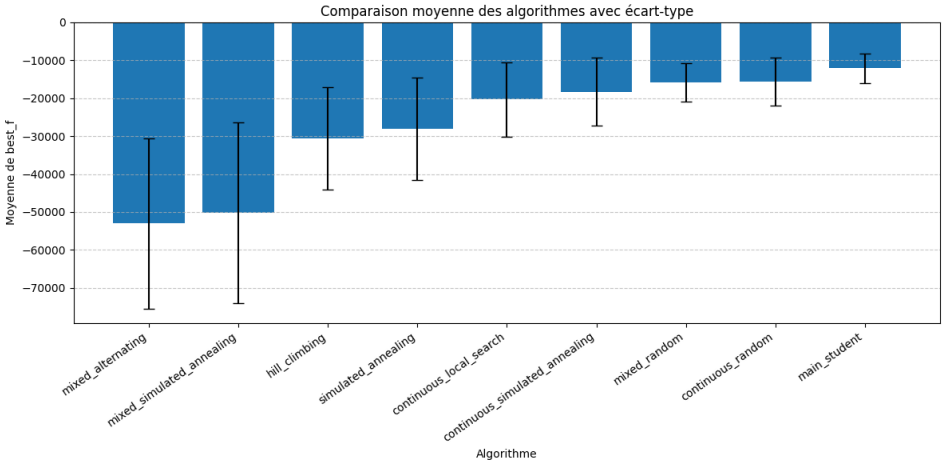


FIGURE 1 – Comparaison moyenne des algorithmes avec écart-type

4.1 Analyse

Le graphique présente la moyenne de la meilleure fitness obtenue pour chaque algorithme, avec des barres d'erreur correspondant à l'écart-type sur les exécutions.

Les résultats montrent que les méthodes mixtes sont les plus performantes. L'algorithme **mixed_alternating** obtient la meilleure moyenne, suivi de près par **mixed_simulated_annealing**. Cela confirme l'intérêt d'optimiser simultanément les variables binaires et continues du problème mixQUBO.

Les méthodes combinatoires seules, comme **hill_climbing** et **simulated_annealing**, obtiennent des résultats intermédiaires. Elles exploitent efficacement la structure binaire du problème, mais ignorent l'optimisation des variables continues, ce qui limite leur performance.

Les méthodes continues seules, comme **continuous_random**, **continuous_local_search** et **continuous_simulated_annealing**, sont moins performantes. Cela indique que la partie binaire joue un rôle important dans la structure du problème.

Les méthodes aléatoires donnent également des résultats plus faibles, car elles n'exploitent pas suffisamment le voisinage des solutions.

Les écarts-types montrent une variabilité plus importante pour les méthodes mixtes. Cette variabilité est cohérente avec le fait que ces méthodes explorent un espace de recherche plus grand, combinant à la fois variables binaires et continues.

Ainsi, les résultats confirment que les approches hybrides sont les plus adaptées pour résoudre efficacement le problème mixQUBO.

5 Conclusion

Ce projet a permis de comparer plusieurs méthodes d'optimisation sur le problème mixQUBO.

Les résultats expérimentaux montrent clairement que les méthodes hybrides sont les plus efficaces. En particulier, **mixed_alternating** obtient les meilleures performances moyennes, suivi de **mixed_simulated_annealing**.

Les méthodes purement combinatoires restent intéressantes, mais elles sont limitées par le fait que les variables continues ne sont pas optimisées. Inversement, les méthodes purement continues ne suffisent pas à capturer la structure principale du problème.

L'optimisation mixte apparaît donc comme l'approche la plus adaptée pour ce type de problème.

5.1 Perspectives

Plusieurs améliorations peuvent être envisagées :

— ajuster automatiquement les paramètres des algorithmes
— améliorer le refroidissement du recuit simulé
— utiliser des voisinages plus riches
— combiner les méthodes avec des stratégies de restart plus avancées
— tester des métaheuristiques plus sophistiquées comme CMA-ES ou des méthodes hybrides adaptatives